

Project Report

Database Management Systems

(Course code: 25TU03MJM0)



Submitted by:

Shrestha Mal

ID: 25WU0102258

Specialization: Artificial Intelligence and Machine Learning

Section: Whales

Faculty: Dr. Kiran Mayee Adavala

(B. Tech 2025-29)

School of Technology

Submitted on: 06/09/2026

Design and Implementation of a Database Management System for Event Registration and Venue Scheduling System

Abstract

Universities and colleges run many events every year, including seminars, workshops, cultural programmes and conferences. When these are managed manually, the same problems repeat: a venue is booked for two sessions at the same time, a participant registers twice for the same event, attendance is recorded inconsistently, and certificates are issued without checking whether the participant attended enough sessions.

This project presents the design and implementation of a relational database system, the **Event Registration and Venue Scheduling System**, that addresses these problems. The database is built in MySQL 8 and normalized to Third Normal Form (3NF). It consists of 13 tables covering categories, events, coordinators, venues, speakers, sessions, participants, registrations, payments, attendance, certificates and feedback. Business rules such as no duplicate registrations, non-negative payments, valid attendance status, rating limits and venue slot uniqueness are enforced as database constraints rather than left to application code.

The database was populated with realistic sample data, including edge cases such as a free event with no payments and a cancelled registration. Ten SQL queries (joins, aggregation, nested subqueries and a self-join) and four views provide operational reports on seats, registrations, attendance, certificate eligibility, revenue, speaker schedules and feedback. A web-based user interface built with Python Flask and mysql-connector-python connects to the database and supports insertion, deletion and viewing of records, and displays the database's own error messages when a constraint is violated.

Keywords: DBMS, MySQL, normalization, 3NF, constraints, SQL views, Flask, event management.

Table of Contents

1. Introduction and Problem Statement
 - 1.1 Introduction
 - 1.2 Problem statement
2. Objectives and Scope
 - 2.1 Objectives
 - 2.2 Users of the system
 - 2.3 Scope
3. Software and Hardware Requirements
 - 3.1 Software requirements
 - 3.2 Hardware requirements
4. ER Diagram
 - 4.1 Key relationships
5. Relational Schema and Normalization
 - 5.1 Relational schema
 - 5.2 Normalization to Third Normal Form
6. Data Dictionary
 - 6.1 categories
 - 6.2 venues
 - 6.3 speakers
 - 6.4 coordinators
 - 6.5 participants
 - 6.6 events
 - 6.7 event_coordinators
 - 6.8 sessions
 - 6.9 registrations
 - 6.10 payments
 - 6.11 attendance
 - 6.12 certificates
 - 6.13 feedback
7. SQL Commands Used (DDL and DML)
 - 7.1 DDL: creating the database and tables
 - 7.2 DML: inserting data
 - 7.3 Sample data summary
 - 7.4 Sample output
8. Queries with Outputs
 - 8.1 Event seats report (join and aggregation)
 - 8.2 Registration summary (confirmed vs cancelled)
 - 8.3 Attendance report (percentage per participant per event)
 - 8.4 Certificate eligibility report
 - 8.5 Revenue report (join and aggregation)
 - 8.6 Speaker schedule report

- 8.7 Feedback report (average rating per session)
- 8.8 Nested query: participants who attended every session of their event
- 8.9 Nested query: events with above-average revenue
- 8.10 Self-join: venue double-booking check
- 8.11 Views
- 8.12 Query given during Presentation-II

9. UI Design and Screenshots

- 9.1 Screens

10. Implementation Details

- 10.1 Technology stack
- 10.2 Database connectivity
- 10.3 Insert and delete
- 10.4 Reports and constraint handling
- 10.5 How to run

11. Testing

12. Conclusion and Future Enhancements

- 12.1 Conclusion
- 12.2 Future enhancements

Acknowledgement

References

Appendix: GitHub Repository Link

1. Introduction and Problem Statement

1.1 Introduction

Event management in a university involves several parties working together: coordinators who organize events, speakers who deliver sessions, venues that have limited capacity, and participants who register, attend and expect a certificate. Each event can have many sessions spread over several days, and each participant can take part in many events. Tracking this information in spreadsheets or on paper is error-prone and does not scale.

A database management system gives a single, consistent store for all of this information. By modelling events, sessions, venues and registrations as related tables, and by letting the database enforce rules itself, the system makes it impossible to store certain kinds of invalid data in the first place.

1.2 Problem statement

Manual event management breaks down in four main ways:

- **Venue double-booking:** two sessions get assigned to the same venue at overlapping times.
- **Duplicate registration:** a participant registers more than once for the same event.
- **Unreliable attendance:** attendance is not tracked per session, so participation cannot be measured.
- **Unverified certificates:** certificates are issued without checking whether the attendance requirement was met.

The goal of this project is to design and build a normalized relational database, with a working user interface, that automates event setup, session scheduling, venue allocation, participant registration, attendance, payments, certificate eligibility and report generation from start to finish.

2. Objectives and Scope

2.1 Objectives

- **No venue overlap:** no two sessions may share the same venue slot.
- **Capacity awareness:** venue capacity is stored for every venue so seats booked can be compared with available capacity.
- **No duplicate registration:** one participant can have only one registration per event.
- **Attendance-based certificates:** certificate eligibility is derived from the percentage of sessions attended (60% threshold).

- **Non-negative payments:** every recorded payment amount must be greater than or equal to zero.
- **Operational reporting:** reports for seats, registrations, attendance, revenue, speaker schedules and feedback.
- **Usable interface:** a UI connected to MySQL for inserting, deleting and viewing records.

2.2 Users of the system

User	Role in the system
Admin / Coordinator	Creates events, schedules sessions, allocates venues and manages records.
Speaker	Is assigned to sessions; the speaker schedule is generated as a report.
Participant	Registers for events, is tracked for attendance, pays fees where applicable, and receives a certificate.

2.3 Scope

The system covers event creation and categorisation, session scheduling, venue allocation, participant registration, payments, attendance, certificate eligibility, feedback and reporting. Out of scope for this version are user login and role-based access, online payment gateways, automatic e-mail notifications and generation of certificate documents.

3. Software and Hardware Requirements

3.1 Software requirements

Component	Details
Database	MySQL 8.0 or later (CHECK constraints require 8.0.16+)
Database tool	MySQL Workbench and MySQL command-line client
Programming language	Python 3
Web framework	Flask
DB connector	mysql-connector-python
Front end	HTML and CSS rendered through Flask templates
Browser	Any modern web browser
Code editor	VS Code or any text editor
Operating system	Windows 10/11 (also works on Linux and macOS)

4.1 Key relationships

No.	Relationship	Meaning
1	Category (1) to Events (M)	One category classifies many events
2	Event (1) to Sessions (M)	An event is made up of multiple sessions
3	Venue (1) to Sessions (M); Speaker (1) to Sessions (M)	Each session has exactly one venue and one speaker
4	Event (M) to Coordinator (N)	Resolved through the event_coordinators junction table
5	Participant (1) to Registrations (M); Event (1) to Registrations (M)	A registration links one participant to one event
6	Registration (1) to Payment (0 or 1)	Free events have no payment row
7	Registration (1) to Attendance (M); Session (1) to Attendance (M)	One attendance row per session checked
8	Registration (1) to Certificate (0 or 1)	Issued only if the attendance threshold is met
9	Registration (1) to Feedback (M); Session (1) to Feedback (M)	Feedback is given per session

5. Relational Schema and Normalization

5.1 Relational schema

Primary keys are shown in bold. Foreign keys are marked (FK).

Table	Schema
categories	(category_id, category_name)
venues	(venue_id, venue_name, capacity, location)
speakers	(speaker_id, full_name, expertise, contact)
coordinators	(coordinator_id, full_name, contact)
participants	(participant_id, full_name, email, phone)
events	(event_id, event_name, category_id (FK), start_date, end_date)
event_coordinators	(event_id (FK), coordinator_id (FK))
sessions	(session_id, event_id (FK), venue_id (FK), speaker_id (FK), session_date, start_time, end_time)
registrations	(registration_id, participant_id (FK), event_id (FK), registration_date, status)
payments	(payment_id, registration_id (FK), amount, payment_date, mode)
attendance	(attendance_id, registration_id (FK), session_id (FK), status)
certificates	(certificate_id, registration_id (FK), issue_date, eligibility_status)
feedback	(feedback_id, registration_id (FK), session_id (FK), rating, comments)

5.2 Normalization to Third Normal Form

First Normal Form (1NF). Every column holds a single atomic value and there are no repeating groups. For example, a participant's several registrations are stored as separate rows in registrations, not as a list inside the participants table.

Second Normal Form (2NF). Every non-key column depends on the whole primary key. All tables use a single-column surrogate key, except event_coordinators, which has no non-key attributes, so no partial dependency can exist.

Third Normal Form (3NF). No non-key column depends on another non-key column. A session stores venue_id, not the venue's name or capacity, so a change in a venue's capacity is made in one row of venues instead of in every session. In the same way, sessions store speaker_id and event_id, and registrations store participant_id and event_id.

Many-to-many relationship. Events and coordinators are related many-to-many, since one event can have several coordinators and one coordinator can manage several events. The junction table event_coordinators(event_id, coordinator_id) with a composite primary key resolves this.

Design note. Attendance percentage and certificate eligibility are derived from attendance rows and are computed by queries and views instead of being duplicated across tables; the certificates table records the eligibility decision and issue date.

6. Data Dictionary

Table structure, data types and constraints for all 13 tables, taken from the DDL script. PK = primary key, FK = foreign key, AI = auto increment.

6.1 categories

Column	Data type	Constraints
category_id	INT	PK, AI
category_name	VARCHAR(50)	NOT NULL, UNIQUE

6.2 venues

Column	Data type	Constraints
venue_id	INT	PK, AI
venue_name	VARCHAR(100)	NOT NULL
capacity	INT	NOT NULL, CHECK (capacity > 0)
location	VARCHAR(150)	NULL allowed

6.3 speakers

Column	Data type	Constraints
speaker_id	INT	PK, AI
full_name	VARCHAR(100)	NOT NULL
expertise	VARCHAR(150)	NULL allowed
contact	VARCHAR(50)	NULL allowed

6.4 coordinators

Column	Data type	Constraints
coordinator_id	INT	PK, AI
full_name	VARCHAR(100)	NOT NULL
contact	VARCHAR(50)	NULL allowed

6.5 participants

Column	Data type	Constraints
participant_id	INT	PK, AI
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
phone	VARCHAR(15)	NULL allowed

6.6 events

Column	Data type	Constraints
event_id	INT	PK, AI
event_name	VARCHAR(150)	NOT NULL
category_id	INT	FK to categories, NOT NULL
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL, CHECK (end_date >= start_date)

6.7 event_coordinators

Column	Data type	Constraints
event_id	INT	PK, FK to events (ON DELETE CASCADE)
coordinator_id	INT	PK, FK to coordinators

6.8 sessions

Column	Data type	Constraints
session_id	INT	PK, AI
event_id	INT	FK to events (ON DELETE CASCADE), NOT NULL
venue_id	INT	FK to venues, NOT NULL
speaker_id	INT	FK to speakers, NOT NULL
session_date	DATE	NOT NULL

Column	Data type	Constraints
start_time	TIME	NOT NULL
end_time	TIME	NOT NULL, CHECK (end_time > start_time)
(table rule)		UNIQUE (venue_id, session_date, start_time)

6.9 registrations

Column	Data type	Constraints
registration_id	INT	PK, AI
participant_id	INT	FK to participants, NOT NULL
event_id	INT	FK to events (ON DELETE CASCADE), NOT NULL
registration_date	DATE	NOT NULL
status	VARCHAR(20)	NOT NULL, DEFAULT confirmed, CHECK in (confirmed, cancelled, waitlisted)
(table rule)		UNIQUE (participant_id, event_id)

6.10 payments

Column	Data type	Constraints
payment_id	INT	PK, AI
registration_id	INT	FK to registrations (ON DELETE CASCADE), NOT NULL, UNIQUE
amount	DECIMAL(10,2)	NOT NULL, CHECK (amount >= 0)
payment_date	DATE	NOT NULL
mode	VARCHAR(20)	NOT NULL, CHECK in (cash, upi, card, netbanking)

6.11 attendance

Column	Data type	Constraints
attendance_id	INT	PK, AI
registration_id	INT	FK to registrations (ON DELETE CASCADE), NOT NULL
session_id	INT	FK to sessions (ON DELETE CASCADE), NOT NULL
status	VARCHAR(10)	NOT NULL, CHECK in (present, absent)
(table rule)		UNIQUE (registration_id, session_id)

6.12 certificates

Column	Data type	Constraints
certificate_id	INT	PK, AI
registration_id	INT	FK to registrations (ON DELETE CASCADE), NOT NULL, UNIQUE
issue_date	DATE	NOT NULL
eligibility_status	VARCHAR(15)	NOT NULL, CHECK in (eligible, not eligible)

6.13 feedback

Column	Data type	Constraints
feedback_id	INT	PK, AI
registration_id	INT	FK to registrations (ON DELETE CASCADE), NOT NULL
session_id	INT	FK to sessions (ON DELETE CASCADE), NOT NULL
rating	INT	NOT NULL, CHECK (rating BETWEEN 1 AND 5)
comments	VARCHAR(500)	NULL allowed
(table rule)		UNIQUE (registration_id, session_id)

Indexes: idx_sessions_event (sessions.event_id), idx_sessions_venue_date (sessions.venue_id, session_date), idx_registrations_event (registrations.event_id), idx_participants_email (participants.email), idx_attendance_registration (attendance.registration_id).

7. SQL Commands Used (DDL and DML)

7.1 DDL: creating the database and tables

The complete script is in 01_DDL.sql. Representative commands:

```
DROP DATABASE IF EXISTS event_registration_db;
CREATE DATABASE event_registration_db;
USE event_registration_db;

CREATE TABLE registrations (
    registration_id INT AUTO INCREMENT PRIMARY KEY,
    participant_id INT NOT NULL,
    event_id INT NOT NULL,
    registration_date DATE NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'confirmed',
    CONSTRAINT fk_reg_participant FOREIGN KEY (participant_id)
        REFERENCES participants(participant_id),
    CONSTRAINT fk_reg_event FOREIGN KEY (event_id)
        REFERENCES events(event_id) ON DELETE CASCADE,
    CONSTRAINT uq_participant_event UNIQUE (participant_id, event_id),
    CONSTRAINT chk_status CHECK
        (status IN ('confirmed', 'cancelled', 'waitlisted'))
);

CREATE TABLE payments (
    payment_id INT AUTO INCREMENT PRIMARY KEY,
    registration_id INT NOT NULL UNIQUE,
    amount DECIMAL(10,2) NOT NULL,
    payment_date DATE NOT NULL,
    mode VARCHAR(20) NOT NULL,
    CONSTRAINT fk_payments_registration FOREIGN KEY (registration_id)
        REFERENCES registrations(registration_id) ON DELETE CASCADE,
    CONSTRAINT chk_amount_nonneg CHECK (amount >= 0),
    CONSTRAINT chk_mode CHECK (mode IN ('cash', 'upi', 'card', 'netbanking'))
);

CREATE INDEX idx_sessions_venue_date ON sessions(venue_id, session_date);
```

7.2 DML: inserting data

The complete script is in 02_SampleData.sql.

```
INSERT INTO categories (category_name) VALUES
('Seminar'), ('Workshop'), ('Cultural'), ('Conference');

INSERT INTO events (event name, category id, start date, end date) VALUES
('TechFest 2026', 4, '2026-09-10', '2026-09-12'),
('AI & Cloud Workshop', 2, '2026-09-15', '2026-09-15'),
('Cultural Night', 3, '2026-09-20', '2026-09-20');

INSERT INTO registrations (participant id, event id, registration date, status) VALUES
(1, 1, '2026-08-25', 'confirmed'),
(8, 1, '2026-08-30', 'cancelled'); -- edge case: cancelled

INSERT INTO payments (registration_id, amount, payment_date, mode) VALUES
(4, 500.00, '2026-08-27', 'upi'),
(6, 100.00, '2026-08-28', 'cash');
```

7.3 Sample data summary

Table	Rows
categories	4
venues	4
speakers	4
coordinators	3
participants	8
events	3
event_coordinators	4
sessions	5
registrations	9
payments	5
attendance	14
certificates	8
feedback	7

The data includes deliberate edge cases: TechFest registrations are free and so have no payment row, and one registration (participant 8, TechFest) is cancelled. These make sure queries and constraints are tested on awkward cases and not only the normal path.

7.4 Sample output

SQL	Output
SELECT COUNT(*) FROM participants;	8
SELECT COUNT(*) FROM registrations;	9
SHOW TABLES;	13 tables and 4 views (17 rows)

```
mysql> USE event_registration_db; SHOW TABLES; SELECT COUNT(*) FROM participants;
Database changed
+-----+
| Tables_in_event_registration_db |
+-----+
| attendance                      |
| categories                      |
| certificates                    |
| coordinators                   |
| event_coordinators              |
| events                         |
| feedback                       |
| participants                    |
| payments                       |
| registrations                   |
| sessions                       |
| speakers                       |
| venues                         |
| vw_attendance_summary           |
| vw_event_revenue                |
| vw_event_seats                 |
| vw_speaker_schedule            |
+-----+
17 rows in set (0.018 sec)

+-----+
| COUNT(*) |
+-----+
|         8 |
+-----+
1 row in set (0.006 sec)
```

Figure 7.1: Database created and populated

8. Queries with Outputs

The full set is in 03_Queries_and_Views.sql. Outputs below are for the sample data in Section 8.

8.1 Event seats report (join and aggregation)

```
SELECT e.event_name,
       COUNT(DISTINCT r.registration_id) AS seats_booked,
       SUM(v.capacity) AS total_capacity_across_sessions
FROM events e
JOIN registrations r ON r.event_id = e.event_id AND r.status = 'confirmed'
JOIN sessions s ON s.event_id = e.event_id
JOIN venues v ON v.venue_id = s.venue_id
GROUP BY e.event_id, e.event_name;
```

event_name	seats_booked	total_capacity_across_sessions
TechFest 2026	3	660
AI & Cloud Workshop	3	120
Cultural Night	2	500

8.2 Registration summary (confirmed vs cancelled)

```
SELECT e.event_name,
       SUM(CASE WHEN r.status='confirmed' THEN 1 ELSE 0 END) AS confirmed_count,
       SUM(CASE WHEN r.status='cancelled' THEN 1 ELSE 0 END) AS cancelled_count,
       COUNT(*) AS total_registrations
FROM registrations r JOIN events e ON e.event_id = r.event_id
GROUP BY e.event_id, e.event_name;
```

event_name	confirmed_count	cancelled_count	total_registrations
TechFest 2026	3	1	4
AI & Cloud Workshop	3	0	3
Cultural Night	2	0	2

8.3 Attendance report (percentage per participant per event)

```
SELECT p.full_name, e.event_name,
       COUNT(a.attendance_id) AS sessions_recorded,
       SUM(CASE WHEN a.status='present' THEN 1 ELSE 0 END) AS sessions_present,
       ROUND(100.0 * SUM(CASE WHEN a.status='present' THEN 1 ELSE 0 END)
            / COUNT(a.attendance_id), 1) AS attendance_pct
FROM registrations r
JOIN participants p ON p.participant_id = r.participant_id
JOIN events e ON e.event_id = r.event_id
JOIN attendance a ON a.registration_id = r.registration_id
GROUP BY r.registration_id, p.full_name, e.event_name;
```

full_name	event_name	sessions_recorded	sessions_present	attendance_pct
Aditya Kumar	TechFest 2026	3	2	66.7
Bhavna Roy	TechFest 2026	3	1	33.3
Chirag Patel	TechFest 2026	3	2	66.7
Divya Menon	AI & Cloud Workshop	1	1	100.0
Esha Gupta	AI & Cloud Workshop	1	1	100.0
Farhan Ali	Cultural Night	1	1	100.0
Gauri Joshi	Cultural Night	1	1	100.0
Aditya Kumar	AI & Cloud Workshop	1	1	100.0

8.4 Certificate eligibility report

```
SELECT p.full_name, e.event_name, c.eligibility_status, c.issue_date
FROM certificates c
JOIN registrations r ON r.registration_id = c.registration_id
JOIN participants p ON p.participant_id = r.participant_id
JOIN events e ON e.event_id = r.event_id
ORDER BY e.event_name, c.eligibility_status;
```

full name	event name	eligibility status	issue date
Divya Menon	AI & Cloud Workshop	eligible	2026-09-16
Esha Gupta	AI & Cloud Workshop	eligible	2026-09-16
Aditya Kumar	AI & Cloud Workshop	eligible	2026-09-16
Farhan Ali	Cultural Night	eligible	2026-09-21
Gauri Joshi	Cultural Night	eligible	2026-09-21
Aditya Kumar	TechFest 2026	eligible	2026-09-13
Chirag Patel	TechFest 2026	eligible	2026-09-13
Bhavna Roy	TechFest 2026	not eligible	2026-09-13

Rows within the same event and status may appear in a different order in MySQL.

8.5 Revenue report (join and aggregation)

```
SELECT e.event_name, COUNT(pay.payment_id) AS paid_registrations,
       SUM(pay.amount) AS total_revenue
FROM payments pay
JOIN registrations r ON r.registration_id = pay.registration_id
JOIN events e ON e.event_id = r.event_id
GROUP BY e.event_id, e.event_name
ORDER BY total_revenue DESC;
```

event_name	paid_registrations	total_revenue
AI & Cloud Workshop	3	1500.00
Cultural Night	2	200.00

TechFest has no payment rows because it is free, so it does not appear.

8.6 Speaker schedule report

```
SELECT sp.full_name AS speaker_name, e.event_name, v.venue_name,
       s.session_date, s.start_time, s.end_time
FROM sessions s
JOIN speakers sp ON sp.speaker_id = s.speaker_id
JOIN events e ON e.event_id = s.event_id
JOIN venues v ON v.venue_id = s.venue_id
ORDER BY sp.full_name, s.session_date, s.start_time;
```

speaker_name	event_name	venue_name	session_date	start_time	end_time
Dr. Ananya Rao	TechFest 2026	Auditorium A	2026-09-10	10:00:00	11:30:00
Dr. Ananya Rao	AI & Cloud Workshop	Seminar Hall B	2026-09-15	09:30:00	12:30:00
Mr. Karan Mehta	TechFest 2026	Auditorium A	2026-09-11	10:00:00	11:30:00
Ms. Priya Sen	Cultural Night	Open Air Theatre	2026-09-20	18:00:00	20:00:00
Prof. Leela Nair	TechFest 2026	Conference Room C	2026-09-12	14:00:00	15:30:00

8.7 Feedback report (average rating per session)

```
SELECT e.event_name, sp.full_name AS speaker_name, s.session_date,
       ROUND(AVG(f.rating), 2) AS avg_rating, COUNT(f.feedback_id) AS feedback_count
FROM feedback f
JOIN sessions s ON s.session_id = f.session_id
JOIN speakers sp ON sp.speaker_id = s.speaker_id
JOIN events e ON e.event_id = s.event_id
GROUP BY s.session_id, e.event_name, sp.full_name, s.session_date
ORDER BY avg_rating DESC;
```

event_name	speaker_name	session_date	avg_rating	feedback_count
TechFest 2026	Dr. Ananya Rao	2026-09-10	5.00	1
TechFest 2026	Mr. Karan Mehta	2026-09-11	4.50	2
AI & Cloud Workshop	Dr. Ananya Rao	2026-09-15	4.50	2
Cultural Night	Ms. Priya Sen	2026-09-20	4.50	2

8.8 Nested query: participants who attended every session of their event

```
SELECT p.full_name, e.event_name
FROM registrations r
JOIN participants p ON p.participant_id = r.participant_id
JOIN events e ON e.event_id = r.event_id
WHERE r.status = 'confirmed'
AND NOT EXISTS (SELECT 1 FROM attendance a
                 WHERE a.registration_id = r.registration_id
                 AND a.status = 'absent');
```

full_name	event_name
Divya Menon	AI & Cloud Workshop
Esha Gupta	AI & Cloud Workshop
Farhan Ali	Cultural Night
Gauri Joshi	Cultural Night
Aditya Kumar	AI & Cloud Workshop

8.9 Nested query: events with above-average revenue

```
SELECT e.event_name, SUM(pay.amount) AS revenue
FROM events e
JOIN registrations r ON r.event_id = e.event_id
JOIN payments pay ON pay.registration_id = r.registration_id
GROUP BY e.event_id, e.event_name
HAVING SUM(pay.amount) > (
    SELECT AVG(event_revenue) FROM (
        SELECT SUM(pay2.amount) AS event_revenue
        FROM registrations r2
        JOIN payments pay2 ON pay2.registration_id = r2.registration_id
        GROUP BY r2.event_id) AS sub);
```

event_name	revenue
AI & Cloud Workshop	1500.00

The average of the per-event revenues (1500 and 200) is 850, so only the workshop is above it.

8.10 Self-join: venue double-booking check

```
SELECT s1.session_id AS session_a, s2.session_id AS session_b, s1.venue_id
FROM sessions s1
JOIN sessions s2
ON s1.venue_id = s2.venue_id AND s1.session_id < s2.session_id
AND s1.session_date = s2.session_date
AND s1.start_time < s2.end_time AND s2.start_time < s1.end_time;
```

session a	session b	venue id
(empty set: 0 rows)		

Zero rows means no two sessions overlap in the same venue.

8.11 Views

Four views save the reporting queries so that the front end can simply run `SELECT * FROM` the view instead of repeating joins.

View	Purpose	Output on sample data
vw_event_seats	Confirmed seats per event	TechFest 2026: 3; AI & Cloud Workshop: 3; Cultural Night: 2
vw_event_revenue	Total revenue per event (0 if none)	TechFest 2026: 0.00; AI & Cloud Workshop: 1500.00; Cultural Night: 200.00
vw_attendance_summary	Attendance percentage per registration	Same figures as Query 3 for every registration
vw_speaker_schedule	Full schedule per speaker	Same rows as Query 6

8.12 Query given during Presentation-II

Display the coordinators, the sessions and the attendance of the Cultural Night event.

```
mysql> SELECT c.full_name AS coordinator_name,
->      s.session_date, s.start_time, s.end_time,
->      p.full_name AS participant_name,
->      a.status AS attendance_status
-> FROM events e
-> JOIN event_coordinators ec ON ec.event_id = e.event_id
-> JOIN coordinators c ON c.coordinator_id = ec.coordinator_id
-> JOIN sessions s ON s.event_id = e.event_id
-> LEFT JOIN attendance a ON a.session_id = s.session_id
-> LEFT JOIN registrations r ON r.registration_id = a.registration_id
-> LEFT JOIN participants p ON p.participant_id = r.participant_id
-> WHERE e.event_name = 'Cultural Night';
```

coordinator_name	session_date	start_time	end_time	participant_name	attendance_status
Arjun Das	2026-09-20	18:00:00	20:00:00	Farhan Ali	present
Arjun Das	2026-09-20	18:00:00	20:00:00	Gauri Joshi	present

2 rows in set (0.310 sec)

9. UI Design and Screenshots

The user interface is a single-page-style web console. A dark teal sidebar lists every table and a Reports page; the main area shows an insert form at the top of each table page, followed by the table of current rows with a Delete button on each row. Foreign keys are chosen from dropdowns that show readable names (for example the event name), so IDs never have to be typed. Success and error messages appear in green and red banners above the content. The layout is responsive and keyboard navigable.

9.1 Screens

Screen	Function
Table page (any of 11 tables)	View rows, insert a row, delete a row
Reports page	Shows the results of the four views and the attendance-based certificate list (attendance 60% or more)

Event Registration Console

- participants
- events
- sessions
- registrations**
- payments
- attendance
- certificates
- feedback
- venues
- speakers
- coordinators
- categories
- Reports (views)

registrations

Add a record, view all rows, or delete one.

participant_id

Aditya Kumar

event_id

TechFest 2026

registration_date

dd-mm-yyyy

status

confirmed

Add registration

9 row(s)

registration_id	participant_id	event_id	registration_date	status	
1	1	1	2026-08-25	confirmed	Delete
2	2	1	2026-08-25	confirmed	Delete
3	3	1	2026-08-26	confirmed	Delete
4	4	2	2026-08-27	confirmed	Delete
5	5	2	2026-08-27	confirmed	Delete
6	6	3	2026-08-28	confirmed	Delete
7	7	3	2026-08-28	confirmed	Delete
8	1	2	2026-08-29	confirmed	Delete
9	8	1	2026-08-30	cancelled	Delete

Figure 9.1: View records

participants

Add a record, view all rows, or delete one.

full_name

Shrestha Mal

email

shrestha.official.005@gmail.com

phone

06289078226

Add participant

8 row(s)

participant_id	full_name	email	phone	
1	Aditya Kumar	aditya.kumar@mail.com	9800000001	Delete
2	Bhavna Roy	bhavna.roy@mail.com	9800000002	Delete
3	Chirag Patel	chirag.patel@mail.com	9800000003	Delete
4	Divya Menon	divya.menon@mail.com	9800000004	Delete
5	Esha Gupta	esha.gupta@mail.com	9800000005	Delete
6	Farhan Ali	farhan.ali@mail.com	9800000006	Delete
7	Gauri Joshi	gauri.joshi@mail.com	9800000007	Delete
8	Harsh Vardhan	harsh.vardhan@mail.com	9800000008	Delete

Figure 9.2: Insert form (Before insert)

Row added to participants.

participants

Add a record, view all rows, or delete one.

9 row(s)

participant_id	full_name	email	phone	
1	Aditya Kumar	aditya.kumar@mail.com	9800000001	Delete
2	Bhavna Roy	bhavna.roy@mail.com	9800000002	Delete
3	Chirag Patel	chirag.patel@mail.com	9800000003	Delete
4	Divya Menon	divya.menon@mail.com	9800000004	Delete
5	Esha Gupta	esha.gupta@mail.com	9800000005	Delete
6	Farhan Ali	farhan.ali@mail.com	9800000006	Delete
7	Gauri Joshi	gauri.joshi@mail.com	9800000007	Delete
8	Harsh Vardhan	harsh.vardhan@mail.com	9800000008	Delete
9	Shrestha Mal	shrestha.offical.005@gmail.com	06289078226	Delete

```
mysql> SELECT COUNT(*) FROM participants;
+-----+
| COUNT(*) |
+-----+
|          9 |
+-----+
1 row in set (0.009 sec)
```

Figure 9.3: After insert

participants

Add a record, view all rows, or delete one.

9 row(s)

participant_id	full_name	email	phone	
1	Aditya Kumar	aditya.kumar@mail.com	9800000001	Delete
2	Bhavna Roy	bhavna.roy@mail.com	9800000002	Delete
3	Chirag Patel	chirag.patel@mail.com	9800000003	Delete
4	Divya Menon	divya.menon@mail.com	9800000004	Delete
5	Esha Gupta	esha.gupta@mail.com	9800000005	Delete
6	Farhan Ali	farhan.ali@mail.com	9800000006	Delete
7	Gauri Joshi	gauri.joshi@mail.com	9800000007	Delete
8	Harsh Vardhan	harsh.vardhan@mail.com	9800000008	Delete
9	Shrestha Mal	shrestha.offical.005@gmail.com	06289078226	Delete

Figure 9.4: Before delete

Deleted 1 row from participants.

participants

Add a record, view all rows, or delete one.

8 row(s)

participant_id	full_name	email	phone	
1	Aditya Kumar	aditya.kumar@mail.com	9800000001	Delete
2	Bhavna Roy	bhavna.roy@mail.com	9800000002	Delete
3	Chirag Patel	chirag.patel@mail.com	9800000003	Delete
4	Divya Menon	divya.menon@mail.com	9800000004	Delete
5	Esha Gupta	esha.gupta@mail.com	9800000005	Delete
6	Farhan Ali	farhan.ali@mail.com	9800000006	Delete
7	Gauri Joshi	gauri.joshi@mail.com	9800000007	Delete
8	Harsh Vardhan	harsh.vardhan@mail.com	9800000008	Delete

```
mysql> SELECT COUNT(*) FROM participants;
+-----+
| COUNT(*) |
+-----+
|          8 |
+-----+
1 row in set (0.009 sec)
```

Figure 9.5: After delete

Rejected by the database: Duplicate entry '1-1' for key 'registrations.uq_participant_event'

registrations

Add a record, view all rows, or delete one.

9 row(s)

registration_id	participant_id	event_id	registration_date	status	
1	1	1	2026-08-25	confirmed	Delete
2	2	1	2026-08-25	confirmed	Delete
3	3	1	2026-08-26	confirmed	Delete
4	4	2	2026-08-27	confirmed	Delete
5	5	2	2026-08-27	confirmed	Delete
6	6	3	2026-08-28	confirmed	Delete
7	7	3	2026-08-28	confirmed	Delete
8	1	2	2026-08-29	confirmed	Delete
9	8	1	2026-08-30	cancelled	Delete

Figure 9.6: Duplicate registration blocked

Rejected by the database: Check constraint 'chk_amount_nonneg' is violated.

payments

Add a record, view all rows, or delete one.

5 row(s)

payment_id	registration_id	amount	payment_date	mode	
1	4	500.00	2026-08-27	upi	Delete
2	5	500.00	2026-08-27	card	Delete
3	6	100.00	2026-08-28	cash	Delete
4	7	100.00	2026-08-28	upi	Delete
5	8	500.00	2026-08-29	upi	Delete

Figure 9.7: Negative payment blocked

Reports

Live results from the SQL views.

Seats booked per event

event_id	event_name	seats_booked
1	TechFest 2026	3
2	AI & Cloud Workshop	3
3	Cultural Night	2

Revenue per event

event_id	event_name	total_revenue
2	AI & Cloud Workshop	1500.00
3	Cultural Night	200.00
1	TechFest 2026	0.00

Attendance summary

registration_id	full_name	event_name	total_sessions	attended	attendance_pct
1	Aditya Kumar	TechFest 2026	3	2	66.7
2	Bhavya Roy	TechFest 2026	3	1	33.3
3	Chirag Patel	TechFest 2026	3	2	66.7
9	Harsh Vardhan	TechFest 2026	0	0	
4	Divya Menon	AI & Cloud Workshop	1	1	100.0
5	Esha Gupta	AI & Cloud Workshop	1	1	100.0
8	Aditya Kumar	AI & Cloud Workshop	1	1	100.0
6	Farhan Ali	Cultural Night	1	1	100.0
7	Gauri Joshi	Cultural Night	1	1	100.0

Speaker schedule

speaker_id	speaker_name	event_name	venue_name	session_date	start_time	end_time
1	Dr. Ananya Rao	TechFest 2026	Auditorium A	2026-09-10	10:00:00	11:30:00
1	Dr. Ananya Rao	AI & Cloud Workshop	Seminar Hall B	2026-09-15	9:30:00	12:30:00
2	Mr. Karan Mehta	TechFest 2026	Auditorium A	2026-09-11	10:00:00	11:30:00
4	Ms. Priya Sen	Cultural Night	Open Air Theatre	2026-09-20	18:00:00	20:00:00
3	Prof. Leela Nair	TechFest 2026	Conference Room C	2026-09-12	14:00:00	15:30:00

Certificate eligible (attendance >= 60%)

registration_id	full_name	event_name	total_sessions	attended	attendance_pct
1	Aditya Kumar	TechFest 2026	3	2	66.7
3	Chirag Patel	TechFest 2026	3	2	66.7
4	Divya Menon	AI & Cloud Workshop	1	1	100.0
5	Esha Gupta	AI & Cloud Workshop	1	1	100.0
8	Aditya Kumar	AI & Cloud Workshop	1	1	100.0
6	Farhan Ali	Cultural Night	1	1	100.0
7	Gauri Joshi	Cultural Night	1	1	100.0

Figure 9.8: Reports

10. Implementation Details

10.1 Technology stack

Layer	Technology	Role
Presentation	HTML, CSS (Flask templates)	Forms, tables, messages
Application	Python 3 with Flask	Routes for view, insert, delete and reports
Connectivity	mysql-connector-python	Opens a connection and executes SQL
Database	MySQL 8, database event_registration_db	Stores data and enforces constraints

10.2 Database connectivity

The application opens a MySQL connection for every request, runs one SQL statement, commits if it changed data, and closes the connection. Parameters are passed separately from the SQL text (%s placeholders), which protects against SQL injection. Table names come from a fixed list in the code and never from user input.

```
DB = dict(host="localhost", user="root",
          password="...", database="event_registration_db")

def run(sql, params=(), fetch=True):
    con = mysql.connector.connect(**DB)
    try:
        cur = con.cursor()
        cur.execute(sql, params)
        if fetch:
            cols = [c[0] for c in cur.description]
            return cols, cur.fetchall()
        con.commit()
        return cur.rowcount
    finally:
        con.close()
```

10.3 Insert and delete

```
@app.route("/t/<name>/insert", methods=["POST"])
def insert(name):
    pk, fields = TABLES[name]
    names = [f[0] for f in fields]
    vals = [request.form.get(n) or None for n in names]
    sql = f"INSERT INTO {name} ({', '.join(names)}) " \
          f"VALUES ({', '.join(['%s'] * len(names))})"
    try:
        run(sql, vals, fetch=False)
        flash(f"Row added to {name}.", "ok")
    except mysql.connector.Error as e:
        flash(f"Rejected by the database: {e.msg}", "err")
    return redirect(url_for("table", name=name))

@app.route("/t/<name>/delete/<int:pk>", methods=["POST"])
def delete(name, pk):
    pkcol = TABLES[name][0]
    run(f"DELETE FROM {name} WHERE {pkcol} = %s", (pk,), fetch=False)
```

10.4 Reports and constraint handling

The Reports page runs SELECT statements on the four views. When MySQL rejects a statement (for example a duplicate registration, a negative payment or a foreign-key violation), the connector raises an error and the UI shows MySQL's own message, so the rules are visibly enforced by the database and not by the interface.

10.5 How to run

- Run 01_DDL.sql, 02_SampleData.sql and 03_Queries_and_Views.sql in MySQL (once).
- Install the packages: pip install flask mysql-connector-python.
- Set the MySQL password in app.py, then run python app.py.
- Open <http://127.0.0.1:5000> in a browser.

11. Testing

Test cases check that each business rule and each UI operation behaves as designed. The expected result column states what MySQL or the UI is designed to do for each input.

ID	Test case	Input	Expected result
T1	View records	Open the registrations page	9 rows shown, matching SELECT in MySQL
T2	Insert valid participant	New name, unique email	Green message; row appears; SELECT shows 9 participants
T3	Delete a row	Delete the new participant	Green message; row removed; SELECT shows 8 participants
T4	Duplicate registration	Participant 1 for event 1 again	Rejected: duplicate entry for uq_participant_event
T5	Duplicate e-mail	Participant with an existing e-mail	Rejected: duplicate entry on the email UNIQUE key
T6	Negative payment	Amount = -100	Rejected: check constraint chk_amount_nonneg violated
T7	Invalid rating	Feedback rating = 6	Rejected: check constraint chk_rating violated
T8	Session end before start	start 12:00, end 10:00	Rejected: check constraint chk_session_times violated
T9	Venue slot clash	Same venue, date and start time as an existing session	Rejected: duplicate entry for uq_venue_slot
T10	Event end before start	end_date earlier than start_date	Rejected: check constraint chk_event_dates violated
T11	Delete referenced row	Delete a participant who has registrations	Rejected: foreign key constraint fails
T12	Delete event	Delete an event	Its sessions and registrations are removed (ON DELETE CASCADE)
T13	Reports page	Open Reports	All views load; revenue 1500.00 and 200.00 shown
T14	Overlap check query	Run Query 10	Empty set

Test T12 permanently removes data; restore the sample data by re-running the three SQL scripts if it is run.

12. Conclusion and Future Enhancements

12.1 Conclusion

This project delivers a complete, normalized MySQL database for managing events, sessions, venues, registrations, payments, attendance, certificates and feedback, together with a working web interface. The key business rules are enforced by constraints in the database itself, so invalid data such as duplicate registrations or negative payments is rejected no matter which application sends it. Ten queries and four views provide the reports an event office needs, and the UI demonstrates insertion, deletion and viewing against the live database.

12.2 Future enhancements

- **Automatic overlap and capacity checks:** a trigger or stored procedure to reject any session whose time range overlaps another in the same venue, and registrations beyond venue capacity.
- **Automatic certificate eligibility:** a stored procedure that computes attendance percentage and writes the certificate row.
- **Login and roles:** separate access for administrators, coordinators and participants.
- **Online payments and e-mail notifications** for registration confirmation.
- **Certificate generation** as downloadable PDF documents with a verification code.
- **Waitlist handling:** automatically promote waitlisted registrations when a seat becomes free.

Acknowledgement

I would like to express my sincere gratitude to **Dr. Kiran Mayee Adavala** for the guidance, encouragement, and valuable feedback provided during the development of this individual project, “Event Registration and Venue Scheduling System.” Their support helped me strengthen my understanding of database management concepts and apply them to the planning, design, and implementation of the system.

I am grateful for the time and care Dr. Adavala gave in reviewing my work and offering suggestions throughout the project. This guidance helped me approach the project more thoughtfully, address challenges as they arose, and improve the quality of the final report.

I also extend my thanks to the faculty and staff of my department for the academic environment and resources that supported my learning. Finally, I acknowledge everyone whose assistance or encouragement contributed, directly or indirectly, to completing this project.

I accept responsibility for the work presented in this report and for any errors or omissions that may remain.

Shrestha Mal

References

- MySQL 8.0 Reference Manual. Oracle Corporation.
<https://dev.mysql.com/doc/refman/8.0/en/>
- MySQL Connector/Python Developer Guide. Oracle Corporation.
<https://dev.mysql.com/doc/connector-python/en/>
- Flask Documentation. Pallets Projects. <https://flask.palletsprojects.com/>
- Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems. Pearson.
- Silberschatz, A., Korth, H. F. and Sudarshan, S. Database System Concepts. McGraw-Hill.
- Course material and instructions, DBMS Course Project, AIML Whales and AIML Panthers.

Appendix: GitHub Repository Link

Repository: <https://github.com/Shrestha3511/DBMS-Course-Project>

Contents: Presentation 1, Presentation 2, Presentation 3 and Project_Report.